

13th International Symposium “Intelligent Systems” (INTELS’18)

Advanced Cat Swarm Optimization Algorithm in Group Robotics Problem

A.P. Karpenko^{*}, I.A. Leshchev

Bauman Moscow State Technical University, Baumanskaya str., 5, Moscow 105005, Russia

Abstract

Localization of plots of land which have the highest level of radiation, chemical or alternative contamination is one of the typical group robotics objectives. The research aim lies in development, software implementation and performance study of the original robotic group control algorithm based on nature-inspired Cat Swarm Optimization algorithm. This paper proposes Cat Swarm Optimization algorithm description, features of software realization and a vast computational experiment results. The practical value of this work lies in applicability of proposed algorithm for decentralized robotic group control systems synthesis.

© 2019 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0/>)

Peer-review under responsibility of the scientific committee of the 13th International Symposium “Intelligent Systems” (INTELS’18).

Keywords: robot; group robotics; SEMS; decentralized robotic group control; nature-inspired algorithms of global optimization; contaminated plots of land localization.

1. Introduction

From a formal point of view, in this paper we consider a class of problems that can be reduced to localization of extrema of an unknown scalar physical field. Detecting zones of radioactive, chemical, biological or other kinds of terrain contamination, damage caused by malignant algae, turbulence, temperature and salinity of the seas, and other similar problems can be posed in this form [1-4].

The methodological basis of decentralized control in group robotics is bionic approaches. To solve objectives of this class, such population algorithms as a particle swarm, a swarm of bees, an ant colony, etc. [5] are widely used. We use the Cat Swarm Optimization (CSO) algorithm proposed by Shu-Chuan Chu, Pei-Wei Tsai in 2007 [6] as the

^{*} Corresponding author. Tel.: +7-903-190-6833.

E-mail address: apkarpenko@mail.ru

basic swarm algorithm. References to a significant number of works dedicated to swarm robotics (published before 2012 inclusive) are presented in [7-11], parallel implementation for CSO algorithm is proposed in [12]. Publications dedicated to group control of robots on the basis of the CSO algorithm are unknown to the authors.

The choice of the CSO algorithm is due to simplicity of its modification which takes into account peculiarities of hardware implementation of a group of robots. The deep modification of the CSO algorithm proposed in this paper was named ACSO (Advanced CSO).

2. Problem statement

We assume that the search space is two-dimensional. We consider the problem of "strategic" control of robots, – that is, we exclude the "tactical" problem of overcoming obstacles from consideration.

Each of the devices $s_i, i \in [1 : |M|]$ is a mother robot (M-robot) carrying a fixed number $|C|$ of small non-returnable satellite devices $s_{i,j}$ (C-robots); $j \in [1 : |C|]$. These robots can once measure the values of the scalar field $\phi(X) = \phi(x_1, x_2)$ (test) at distances from the robot s_i which do not exceed δ_x for each of the measurements x_1, x_2 – that is, in a square vicinity ϖ_i . Thereby, the total number of tests that C-robots belonging to a given M-robot can perform is limited and equal to $|C| = n_C$. The coordinates of the tests made by C-robots and their results become known to the corresponding M-robot. Each of the M-robots can also perform tests (at their stop points), the total number of which does not exceed $n_M = \hat{t}$.

We assume that the results of the tests of M and C-robots are accurate, that is, the measured values of the function $\phi(X)$ at the test points are not distorted by noise and/or measurement errors. The robots can determine their coordinates x_1, x_2 with an error which is many times less than the value δ_x , so that these errors can be ignored.

We introduce the following notations: $\Pi = \{X \mid X^- \leq X \leq X^+\} \in R^2$ is the investigated region (parallelepiped) of the search space, where the inequalities are understood component by component; $X^- = (x_1^-, x_2^-)$ are the boundaries of the region; t – is the number of the current iteration of the search; C – is the current position of the robot s_i ; $X'_i(t) = X'_i = (x'_{i,1}, x'_{i,2})$ is the position of this robot at the next iteration; X^* – is the desired vector of optimal coordinates X , which delivers the maximum value of the fitness function $\phi(X)$. The length of the diagonal of the parallelepiped Π (the diameter of the set Π) is denoted by d .

We consider the deterministic maximization problem $\max_{X \in \Pi \subset R^2} \phi(X) = \phi(X^*) = \phi^*$.

3. New entities used by the ACSO algorithm

Neighborhood of M-robots. The ASCO algorithm uses the concept of the neighborhood of M-robots in the search space R^2 . The neighborhood topology is defined by the Euclidean norm $\|X_i - X_j\|_E, i, j \in [1 : |S|]$, so that the set $N_i(t) = N_i$ of neighbors of the robot $s_i \in S$ is formed by M-robots whose positions satisfy the condition $\{X_j \mid \|X_i - X_j\|_E \leq r_X, s_j \in S\}, i \in [1 : |S|]$, where r_X is the radius of the vicinity Ω_i .

Trace and extended trace of the M-robot. Trace $Tr_i(t)$ of the M-robot $s_i, i \in [1 : |S|]$ is a collection of all its positions in the search space on iterations $\tau \in [0 : t]$, as well as the corresponding values of the fitness function: $Tr_i(t) = Tr_i = \{(X_i(\tau), \phi_i(\tau)), \tau \in [0 : t]\}$. Similarly, the extended tracing $Tr_i(t)$ of this robot is defined by the expression $Tr_i(t) = \{Tr_i(t) \cup (X_{i,j}, \phi_{i,j}), j \in [1 : n_C(t)]\}$, where $\phi_{i,j} = \phi(X_{i,j})$; $n_C(t) \in [0 : n_C]$ is part of the C-robots $s_{i,j}$ used (dispatched) by the robot s_i by the end of the iteration t .

Perspective search direction. Allow $(X_{i1}, \phi_{i1}) (X_{i2}, \phi_{i2})$ to be two points that are "close" to the region Π_i , and $\phi_{i1} > \phi_{i2}$. The direction defined by these points is called perspective for the region Π_i (Π_i -perspective), if the ray

(X_{i2}, X_{i1}, ∞) crosses the region Π_i (Fig. 1). The number Π_i of perspective directions m_i is determined by the following scheme.

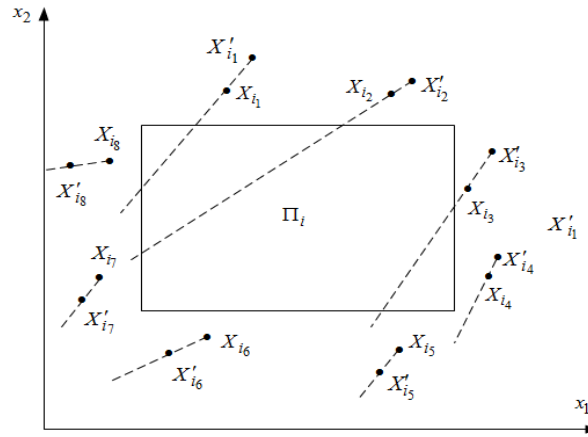


Fig. 1. Determination of perspective directions: $m = 8$; $m_i = 4$; $\phi_{ik} > \phi'_{ik}$, $k \in [1:4]$; $k \in [5:8]$.

1) We find $m = \min(\tilde{m}, |Tr_i(t)|)$ of the points $(X_{i1}, \phi_{i1}), (X_{i2}, \phi_{i2}), \dots, (X_{im}, \phi_{im})$ which belong to the tracing $Tr_i(t)$ and whose distances to the center X_i^C of the region Π_i are minimal. Here $\tilde{m}$ is the free parameter of the algorithm.

2) For each of these points X_{ij} , in the trace $Tr_i(t)$ we find the nearest point (X'_{ij}, ϕ'_{ij}) ; $j \in [1:m]$.

3) We define all Π_i -perspective directions determined by points $(X_{ij}, \phi_{ij}), (X'_{ij}, \phi'_{ij})$; $j \in [1:m]$. The number of these directions is denoted by m_i .

Attractiveness of the region. Each of the regions Π_i , as well as a candidate for the region (see below), has a characteristic called attractiveness a_i . We use the two-criteria rule for determining this value, which has the form of

$$a_i = \lambda \frac{d_i}{d} + (1 - \lambda) \frac{m_i}{m}, \quad \lambda \in [0; 1], \quad (1)$$

where d_i is the region's diameter equal to the length of the diagonal of the rectangle Π_i ; d is the diameter of the search area Π ; m_i , m is the number of Π_i -perspective directions and the total number of directions, respectively; λ – is the weight coefficient (free parameter of the algorithm).

Region and candidates in the search region. The M-robot s_i , $i \in [1:S]$ in its current position $X_i(t)$ can use the region Π_i – one of the nearest unexplored sub regions of the search area Π . The M-robot s_i selects the region Π_i if the set of extended traces of this robot has sufficient power. The selection is performed in three stages (we omit some details).

Stage 1 – definition of candidates for the region Π_i .

1) We sort the coordinate x_1 of all extended traces in the memory of this robot, in ascending order: $x_1^-, x_1^1, x_1^2, \dots, x_1^+$. Similarly, we sort the coordinate x_2 : $x_2^-, x_2^1, x_2^2, \dots, x_2^+$. For simplicity, here and below, we omit the indication that these lists are associated with the robot s_i .

2) We find in the first list a pair of numbers (x_1^k, x_1^{k+1}) for which the value $x_1^{k+1} - x_1^k$ is the maximum. Similarly, we find a pair of numbers (x_2^l, x_2^{l+1}) . As a candidate $\tilde{\Pi}_{i,1}$, we take a rectangle whose vertices are the indicated points. We calculate the diameter $\tilde{d}_{i,1}$ of the rectangle, as well as the coordinates $\tilde{X}_{i,1}^C$ of its center.

3) Similarly, we select the regions $\tilde{\Pi}_{i,2}, \tilde{\Pi}_{i,3}, \dots, \tilde{\Pi}_{i,e}$ and determine their diameters and the coordinates of the centers. The criterion for the end of this process is the condition $\tilde{d}_{i,e} = d(\tilde{\Pi}_{i,e}) < 2\delta_X$.

Stage 2 – narrowing the set of candidates for the region Π_i .

From the set of candidates $\tilde{\Pi}_{i,1}, \tilde{\Pi}_{i,2}, \dots, \tilde{\Pi}_{i,e}$, we exclude sub regions that do not satisfy the restrictions on the speed of the robot's movement: $(\tilde{X}_{i,j}^C - X_i) \in \Pi_\Delta$. The remaining regions are denoted by $\Pi'_{i,1}, \Pi'_{i,2}, \dots, \Pi'_{i,r}$; $r \leq e$.

Stage 3 – selection of the region Π_i from the candidates $\Pi'_{i,1}, \Pi'_{i,2}, \dots, \Pi'_{i,r}$.

1) If the specified set of candidates for the regions is empty (that is, if $r = 0$), then we skip this iteration (leaving the M-robot in its current position).

2) If not ($r > 0$), we determine the attractiveness $a'_{i,j}$ of each of the regions $\Pi'_{i,j}$, and on the basis of these values, according to the roulette wheel rule, we select the region Π_i ; $j \in [1:r]$.

Inadmissible approach of M-robots. In the process of evolution of M-robots $s_{i1}, s_{i2} \in S$ in the search space the situation of their inadmissible approach is possible when the Euclidean distance between these robots is less than $\rho_X : \|X_{i1} - X_{i2}\| < \rho_X$. The rule of behavior of the robots s_{i1}, s_{i2} in this situation has the following form.

1) M-robots s_{i1}, s_{i2} determine the regions Π_{i1}, Π_{i2} and, thereby, their attractiveness a_{i1}, a_{i2} , and exchange the values of these quantities among themselves.

3) If $a_{i1} > a_{i2}$, then we move the robot s_{i1} to the center of the region Π_{i1} (tracing mode), and the robot s_{i2} continues to evolve in the seeking mode. If not, the robot s_{i2} , being in the tracing mode, moves to the center of the region Π_{i2} , and the robot s_{i1} continues to evolve in the seeking mode.

Landing of C-robots. The operation algorithm of the M-robot $s_i \in S$ includes the rules for selecting the points of time $t_{i,1}, t_{i,2}, \dots, t_{i,e}$ and values $n_C(t_{i,1}), n_C(t_{i,2}), \dots, n_C(t_{i,e})$. We use the mediated choice of values $t_{i,1}, t_{i,2}, \dots, t_{i,e}$: at the point of time $t_{i,j}$ the C-robots land, if the value

$$n_C(t_{i,j}) = \begin{cases} \lceil a_i \rceil, & \lceil a_i \rceil \geq n_C^{rem}(t_{i,j}), \\ n_C^{rem}(t_{i,j}), & \text{else,} \end{cases} \quad (2)$$

is different from zero. Here $n_C^{rem}(t_{i,j})$ is the number of C-robots remaining with the M-robot s_i by the point time $t_{i,j}$; $\lceil \cdot \rceil$ is the symbol of the nearest integer. Thus, the landing number of C-robots $n_C(t_{i,j})$ is proportional to the attractiveness a_i of the region Π_i . The formula (2) means that if the given M-robot runs out of C-robots, then only the M-robot continues the seeking.

Inter-robot communication. At each iteration t , neighboring M-robots exchange information with one another. In this case, each of the robots $s_i \in S$ performs the following actions:

1) it transmits its extended trace $Tr_i(t)$ to all robots in the set N_i , as well as extended traces of all the robots which it stores in its memory;

2) it receives similar information from each of the robots in the set N_i .

4. The ACSO algorithm scheme

Initialization of the population is carried out according to the scheme of the original CSO6 algorithm, with the difference that we declare all $|S|$ robots in the state of investigation. We assign the meaning of the required localization accuracy of the maximum of the fitness function $\phi(X)$ to the value δ_X .

The seeking mode of the ACSO algorithm differs from the same mode of the CSO algorithm because in this case not all $\delta_X = \rho_X = \Delta x_i = d/100$, $i = 1, 2$ of the satellite robots participate on the iteration t in the local search but only a part $n_C(t)$ of them, which, in general, is different for each of the M-robots. If the robot s_i drops C-robots landing on iterations $t_{i,1}, t_{i,2}, \dots, t_{i,e}$, then, as the C-robots are non-returnable, the following equality is true: $n_C(t_{i,1}) \cup n_C(t_{i,2}) \cup \dots \cup n_C(t_{i,e}) = n_C$.

The tracing mode of the ACSO algorithm changes cardinally in comparison with the CSO algorithm.

Any of the M-robots can switch to the tracing mode in the following two situations:

- in the seeking mode on the iteration t , it turned out that the values of the fitness function at all points $n_C(t)$ differ by no more than by the value δ_ϕ (the situation is interpreted as localization of one of the contamination sources with accuracy δ_X);
- the Euclidean distance between some two M-robots turned out to be less than the value ρ_X (Fig. 2).

The M-robot s_i in the tracing mode moves to the point with coordinates

$$X_i(t+1) = X_i^C + d_i N_2(0, \sigma), \quad (3)$$

where X_i^C is the center of the region Π_i ; $N_2(0, \sigma)$ is a two-dimensional vector of normal random numbers with zero mean and standard deviation which is equal to σ . Movement takes place only if the region Π_i is large enough, that is, if the inequality $d_i > 2\delta_X$ is true.

5. Software Implementation and Computational Experiment

The software implementation of the ACSO algorithm was performed in the integrated development environment of Microsoft Visual Studio, which allows to connect third-party extensions at various levels. The object-oriented programming language C# is used, which allows to quickly create a graphical user interface and graphics of varying complexity.

The research was carried out in the multi-start mode (the number of starts is 100) with the following values of the free parameters of the algorithm: $|S| = 10$; $c = 1.0$; $\tilde{m} = 8$; $\delta_X = \rho_X = \Delta x_i = d/100$, $i = 1, 2$; $r_X \approx d/10$; $n_C = 25$; $\hat{t} = 100$; $\delta_\phi = 0.01(\phi_{\max} - \phi_{\min})$, where the minimum and maximum possible values of the fitness function $\phi_{\max}, \phi_{\min}$ should be evaluated before the beginning of the solution of the problem; $\sigma \approx d/1000$; restrictions $\Delta X^-, \Delta X^+$ on the speed of the M-robot s_i are determined from the conditions $\Delta x_{i,k}^- = 0.01x_k^-$, $\Delta x_{i,k}^+ = 0.01x_k^+$; $i \in [1 : |S|]$, $k = 1, 2$.

Consider reverse Six-hump camel function as an objective function:

$$\phi(X) = -((4 - 2.1x_1^2 + x_1^4/3)x_1^2 + x_1x_2 + (-4 + 4x_2^2)x_2^2).$$

We examine objective function in $\Pi = \{X \mid x_j^- \leq x_j \leq x_j^+, j = 1, 2\}$ rectangle, where $x_j^- = -5$, $x_j^+ = 5$, $j = 1, 2$.

Proximity of the computed value of the objective function to its exact maximum value with an error not exceeding the value δ_ϕ was used as the criterion of localization.

The algorithm efficiency is estimated by assessment of localization probability of one, two maxima of objective function. Fig. 2 presents estimated probability of localizing the maxima of a function inverse to the two extreme Six-

hump camel function for the number of iterations $\hat{t} = 50, 75, 100$ and parameter values $\lambda = 0, 0.50$. We remember that small values of the value λ intensify search, and values close to one diversify it.

Fig. 2 shows that the ACSO algorithm in all its 100 starts provides, with a specified accuracy, localization of the global maximum of the objective function. Estimated probability of localization of all two global maxima approximately equals 1.

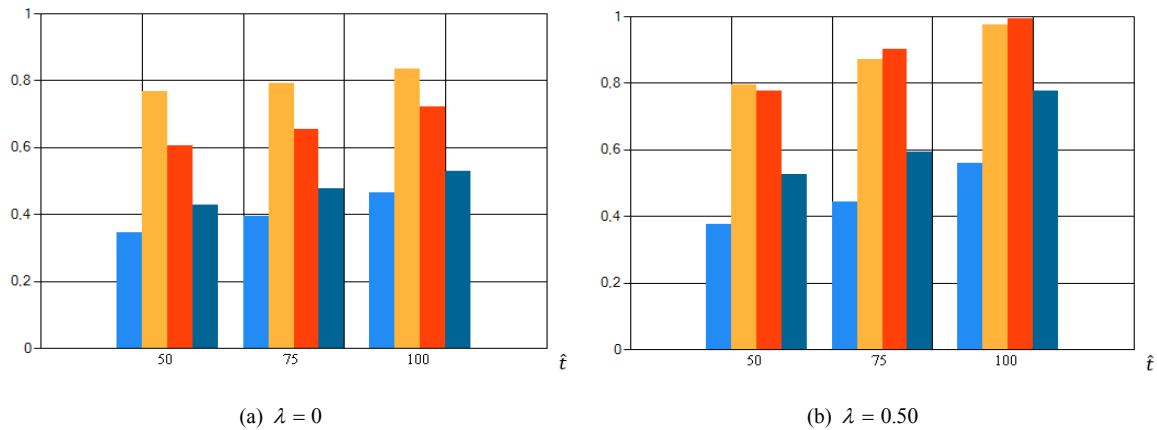


Fig. 2. Estimated probability of localization the maxima of the function that is inverse to Six-hump camel function: blue, orange – one, two global maxima, respectively (the localization criterion based on δ_ϕ); red, blue – one and two maxima respectively (the localization criterion based on δ_x).

6. Conclusion

The deep modification of the CSO algorithm proposed in this paper was named ACSO (Advanced CSO). The ACSO algorithm includes a significant number of free parameters that allow a broad range of balancing diversification and intensification properties of the algorithm. Thereby, ACSO, is, in fact, metaheuristic.

The computational experiment showed high efficiency of proposed algorithmic and program solutions.

We believed that the results of tests of the M- and C-robots are accurate. In the development of the work, the authors propose to investigate the situation when the measurement results are distorted by additive noise or measurement errors with known statistical characteristics. It is also proposed to expand the algorithm for the dimension of the search space equal to three. In addition, it is important to study the effectiveness of the asynchronous variant of the ACSO algorithm.

References

- [1] Pettersson LM, Durand D et al. *Monitoring of Harmful Algal Blooms*. London: Praxis Publishing, 2012.
- [2] White BA, Tsourdos A et al. Contaminant cloud boundary monitoring using UAV sensor swarms. *Proceedings of the AIAA Guidance, Navigation, and Control Conference*. San Francisco, USA. 2005; p. 1037-1043.
- [3] Hayes AT, Martinoli A, Goodman RM. Distributed odor source localization. *IEEE Sensors Journal* 2002;2:3:260-271.
- [4] Lilienthal A, Duckett T. Creating gas concentration grid maps with a mobile robot. *Proceedings of the IEEE/RSJ Conf. on intelligent Robots and Systems*. Las Vegas, USA. 2003; p. 118-123.
- [5] Bo Xing, Wen-Jing Gao. *Innovative Computational Intelligence: A Rough Guide to 134 Clever Algorithms*. Cham: Springer; 2014.
- [6] Chu SC, Tsai PW, Pan JS. *Cat Swarm Optimization*. LNAI 4099, 3(1). Berlin, Heidelberg: Springer-Verlag, p. 854-858.
- [7] Orouskhani M, Mansouri M, Teshnehlab M. *Average-Inertia weighted Cat swarm optimization*. LNCS, Berlin Heidelberg: Springer-Verlag, p. 321-328.
- [8] Orouskhani M, Orouskhani Y et al. A Novel Cat Swarm Optimization Algorithm for Unconstrained Optimization Problems. *IJ Information Technology and Computer Science* 2013;11:32-41.
- [9] Crawford B, Soto R et al. A Binary Cat Swarm Optimization Algorithm for the Non-Unicost Set Covering Problem. *Mathematical Problems in Engineering* 2015;2015-8.
- [10] Kanwar N, Gupta N et al. Improved Cat Swarm Optimization for Simultaneous Allocation of DSTATCOM and DGs in Distribution Systems. *J of Renewable Energy* 2015;2015-10.

- [11] Xiaohua Nie, Wei Wang et al. Chaos Quantum-Behaved Cat Swarm Optimization Algorithm and Its Application in the PV MPPT. *Computational Intelligence and Neuroscience* 2017;**2017**-11.
- [12] Pei-Wei Tsai, Jeng-Shyang Pan et al. Parallel cat swarm optimization. *Proceedings of the Seventh International Conference on Machine Learning and Cybernetics, Kunming, 12-15 July 2008*. **6**:3328-3333.